

3003ICT Group Project Report

Project Title: Indoor Cleaning Robot (Voom-Bot)

Specialisation Option Chosen: Option B - Autonomous Robotics (Webots)

GitHub link: <https://github.com/B4Ashh/Programming-For-Robotics-Assignment-for-GC7>

Video Link:

https://1drv.ms/v/c/def688dcf44d3c99/IQALsy_bQjNASaGcd_WJgbZlARDiCGpiOM4xbYRxWIXRaoo?e=wSuUId

Group Members:

Member	Contribution	S Number
Lucas El-Harrif	Sensor Integration, FSM Logic, design and integration	s5342319
Michelle Murutsi	Navigation, Collision recovery, perception and Safety	s5299221
Tashinga Blessed Nyamadzwo	Grid Mapping and visualisation. Demonstration	s5415355

1. Project Overview

Voom-Bot is an autonomous indoor cleaning robot simulated in Webots using the Pioneer 3-DX platform, designed for lived-in spaces – classrooms, libraries and living rooms.

Because Voom-Bot is a robotic vacuum cleaner, low-speed contact with obstacles is an expected and handled part of how it operates. It navigates a 10x10m arena using a boustrophedon coverage pattern – moving in straight lines, turning 180 degrees (π radians) at each row end and tracing parallel passes to systematically sweep the floor. It takes advantage of the Pioneer 3-DX's 16-channel sonar array, GPS and compass for spatial awareness, and demonstrates coverage path planning, collision detection and recovery, battery return-to-dock and live grid visualisation across four Python modules reflecting a Sense-Think-Act-Map architecture.

2. Technical Requirements Compliance

Voom-Bot satisfies all of the project requirements from the specification. These are as follows:

1. **At least 2 sensor inputs:** Voom-Bot uses the 16-channel sonar array (distance sensing), GPS and compass for pose estimation. These three input channels help it navigate, clean and make decisions.
2. **At least 2 actuator outputs:** The robot drives its left and right wheels independently, which gives it forward motion, turning, spinning in place and reversing – all from just those two outputs.
3. **FSM with a minimum of 4 states:** Our robot implements 6 states – STARTUP, SWEEP, AVOID, PAUSED, RETURN_HOME and DONE. Each one serves a specific purpose that ensures the robot operates safely within its environment.
4. **Multi-condition decision logic:** Every timestep, the FSM evaluates 5 conditions in priority order – person proximity, battery level, sweep completion, predicted collision and reactive sonar proximity. The priority ordering is intentional and we'll explain it in Section 4.
5. **Safety / fail-safe mechanism:** Voom-Bot returns to its home position when the battery drops below 20%. It also has a three-phase obstacle recovery manoeuvre - backup, turn and drive - to clear collisions, and a dual stall detector that kicks in if the robot gets stuck or starts going over already-cleaned floor.
6. **Context-aware behaviour:** Voom-Bot changes what it does based on what's happening in the environment - the battery level changes its navigation goal, the coverage percentage drives its stall detection, and the presence of a person pauses it mid-sweep.

7. **Real-time logic:** All sensing, decision-making and motor commands are completed within a single 64ms simulation timestep, giving Voom-Bot responsive, real-time behaviour throughout the run.

3. System Architecture

Voom-Bot uses a Sense-Think-Act-Map architecture with clear separation between layers:

Sense (sensors.py) – Raw sonar cleaning, zone grouping, person detection, collision severity classification, predictive collision detection

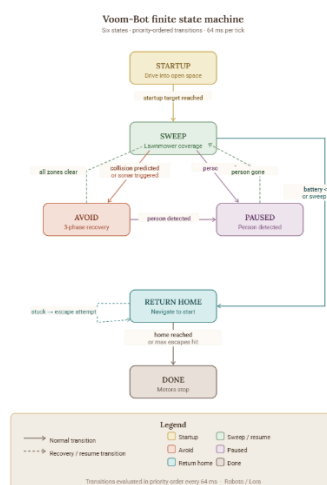
Think (navigation.py) – FSM, boustrophedon path planning, obstacle recovery, waypoint steering

Act (vacuum_controller.py) – Motor commands, stall detection, home escape logic, HTML grid visualisation

Map (grid_map.py) – Occupancy grid, coverage calculation, coordinate conversion, obstacle marking

Data flows unidirectionally: sensors.py → navigation.py → vacuum_controller.py → grid_map.py.

4. Behaviour Model



State	Description	Entry Condition	Exit Condition
STARTUP	Drive away from starting corner	On initialisation	Startup target reached
SWEEP	Execute lawnmower coverage pattern	Startup complete or AVOID cleared	Person detected, battery <20%, obstacle, sweep complete
AVOID	Three-phase obstacle recovery	Collision predicted or sonar triggered	All zones clear
PAUSED	Stop and wait	Person within 0.60m of front sensors	Person gone or 5-tick timeout (0.32s)
RETURN_HOME	Navigate to start position	Battery <20% or sweep complete	Within 0.45m of start, or 8 escapes exhausted
DONE	Hard stop	Home reached or max escapes hit	Terminal

Priority order (evaluated every tick): Person → Battery → Sweep complete → Predicted collision → Sonar → SWEEP. Safety is prioritised over task completion.

Notable behaviours: AVOID chains automatically through backup → turn → forward, skipping to turn if rear is blocked. RETURN_HOME has its own stuck detection with up to 8 escape attempts alternating turn direction. PAUSED has a 5-tick timeout to prevent indefinite blocking.

5. Perception / Sensor Processing

The Pioneer 3-DX provides three input channels — sonar, GPS and compass — all processed in `sensors.py` before reaching the navigation layer.

`sonar_zones()` groups 16 raw readings into 9 named zones (front, front_wide, left, right, side_min etc.) so navigation logic never references raw indices. `clean_readings()` filters values outside 0.02m–5.0m, replacing invalid readings with 5.0m. Person detection uses only `so3` and `so4` (± 10 degrees front-centre) at 0.60m — narrowed from a wider configuration that was triggering on box edges. `collision_severity()` classifies situations as stuck/danger/warning/none for graduated responses. `predict_collision()` simulates 8 steps (512ms) using differential drive kinematics, flagging collisions before physical contact.

Obstacle marking uses GPS position plus actual measured sonar distance rather than polar-to-Cartesian conversion — the Pioneer 3-DX angle offsets in Webots have small discrepancies that placed marks 1–2 cells off the real box face. GPS-based marking is always accurate, and wall cells are never overwritten.

6. Safety and Robustness

1. Battery detection — Triggers `RETURN_HOME` at 20%, reserving capacity for the return journey from any arena position.
2. Collision severity — `collision_severity()` gives graduated responses (steer away / reverse / aggressive recovery) rather than binary stop-and-go.
3. Three-phase recovery — Backup → turn toward open side → forward. Skips to turn phase if rear is blocked.
4. Dual stall detector — Checks position AND coverage every 100 steps. Either frozen for 3 checks fires an escape with alternating turn direction, preventing looping.
5. Person detection with timeout — `PAUSED` exits after 5 ticks (0.32s) even if detection is still firing, preventing indefinite blocking.
6. Hard stop — Both wheel velocities set to zero on `DONE`; main loop breaks.

7. Code Structure

Voom-Bot is built across four Python modules, each with a single defined responsibility:

- **grid_map.py — Data layer** Voom-Bot's memory. Holds a 49×49 occupancy grid (20cm cells, 9.8×9.8m coverage) where every cell is `UNVISITED`, `VISITED`, `OBSTACLE` or `WALL`. `world_to_grid()` converts GPS metres to grid indices with bounds clamping. `mark_visited()` won't overwrite walls or obstacles. `mark_obstacle_ahead()` places marks using GPS position and actual measured sonar distance rather than sonar angle trigonometry — which was inaccurate on the Pioneer 3-DX. `coverage_percent()` excludes walls and obstacles from its denominator so 100% is always mathematically achievable.
- **sensors.py — Perception layer** Voom-Bot's senses. Every function is stateless — pure input-to-output, nothing held between calls. `clean_readings()` filters invalid sonar values (below 0.02m or above 5.0m). `sonar_zones()` groups 16 raw readings into 9 named zones so navigation never references raw indices. `person_nearby()` uses only `so3` and `so4` at 0.60m — narrowed from a wider config that was triggering on box edges. `collision_severity()` returns stuck/danger/warning/none for graduated responses. `predict_collision()` simulates 8 steps (512ms) of future movement using differential drive kinematics — this is what generates the [predict] imminent collision messages.

- **navigation.py — Decision layer** Voom-Bot's brain. `_build_lawnmower()` generates 1,369 waypoints at startup across 37 rows in boustrophedon order. `_steer()` is the proportional heading controller converting angular error to wheel speed difference. `_wall_avoidance_steer()` blends heading control with reactive sonar pushes for RETURN_HOME. `_avoid()` chains the three-phase recovery automatically, skipping to turn phase if the rear is blocked. `step()` is the single FSM entry point called every tick, returning (left_speed, right_speed).
- **vacuum_controller.py — Integration layer** Where everything comes together. Initialises all hardware with a 20-tick warmup, then runs the main loop — SENSE → MAP → SAFETY → ACT every 64ms. Marks a 3×3 visited footprint from GPS every tick. The dual stall detector fires every 100 steps if position or coverage is frozen across 3 consecutive checks, skipping 18 waypoints and running a 160-step alternating escape. Return-home stuck detection is separate — attempt counter persists across cycles, resets only on genuine 0.5m progress, alternates turn direction per attempt, stops gracefully after 8 failures. `export_grid_html()` writes an auto-refreshing grid page every 300 steps.

8. Key Code Snippets

1. Grid and visited cell marking:

```
1) # Mark robot footprint (3×3 cells = 60cm × 60cm) as visited
2) row, col = gmap.world_to_grid(rx, ry)
3) for dr in [-1, 0, 1]:
4)     for dc in [-1, 0, 1]:
5)         gmap.mark_visited(row + dr, col + dc)
```

2. Multi-condition Finite state Machine Transition:

```
if person_near:                                # 1. Safety — always first
    self.state = PAUSED

elif battery_frac < BATTERY_LOW:                # 2. Fail-safe — go home
    self.state = RETURN_HOME

elif self.wp_index >= len(self.waypoints):      # 3. Sweep complete — go home
    self.state = RETURN_HOME

elif predict_collision(rx, ry, rtheta,          # 4. Predictive collision
                      last_lv, last_rv, zones['raw'])[0]:
    self.grid.mark_obstacle_ahead(rx, ry, rtheta, zones)
    self.state = AVOID

elif (zones['front_wide'] < 0.28                # 5. Reactive sonar
      or zones['left'] < 0.20
      or zones['right'] < 0.20
      or zones['side_min'] < 0.14):
    self.grid.mark_obstacle_ahead(rx, ry, rtheta, zones)
    self.state = AVOID

# 6. Default — keep sweeping
```

3. Stall detection and Escape

```
# Checked every 100 steps during SWEEP only
moved = math.hypot(rx - stall_last_x, ry - stall_last_y)
coverage_gain = abs(coverage - last_coverage)

# Dual condition — fires if EITHER position OR coverage is frozen
```

```

if moved < 0.25 or coverage_gain < 0.05:
    stall_ticks += 1

if stall_ticks >= STALL_LIMIT:           # 3 ticks = ~19 seconds
    nav.wp_index      += ROW_SKIP         # skip ~18 waypoints
    force_turn_counter = FORCE_TURN_STEPS * 2
    force_turn_dir     *= -1              # alternate direction each escape
    stall_ticks        = 0

```

9. Results / Demonstration Summary

Based on the Run used for the demonstration of this project, Voombot successfully does the following:

- Cleans 83.2% of its floor arena
- Builds and follows a lawnmower path
- Stall detections when cleaning visited areas and recovers the robot
- The battery fail safe is triggered on time and the robot has enough capacity to make its trip home
- The robot is capable of navigating its way home

The robot is capable of doing the following but needs improvement:

- Stall recovery – It works every time but the robot does it often. This implies that the robot is either walking the same path multiple times or that the robot is getting stuck often. Regardless, it results in the robot wasting resources

The robot fails at:

- Completely cleaning the demonstration arena
- Mapping out obstacles on the grid – it is currently inaccurate
- Replanning around known obstacles - because the waypoints are calculated on startup, the robot does not think to update the fact that there is a obstacle in the path it wants to travel on.